

This is unreleased documentation for alphaTab **1.5.0-alpha.1342** 🚧.

[API Reference](#)[Data Model](#)

Data Model

The core of alphaTab is the data model. It contains all the information about a music piece like the title, author, instruments contained down to the individual notes and their effects.

Everything in alphaTab is built around this data model:

- Importers fill them from the supported file formats
- The ScoreRenderer generates a visual tree with glyphs to render the notes
- The MidiFileGenerator produces a MidiFile from the notes for audio playback.

The data model of alphaTab tries to avoid storing any **visual** information. It is a semantic description of the music sheet on which tracks, bars, notes etc. exist and what effects they have.

This documentation will not go through all the individual properties of the data model but rather describe the rough structure and where to find what. We recommend to have a look at the code documentation (TS doc, XML doc).

Direct modification of this data model is usually not easily possible. The importers do the complicated job there to build the right structure and wrap-up the consistency of the model. While some modifications might be possible and are reflected after a re-rendering, others might fully break the rendering pipeline due to the inconsistencies or unexpected values.

Main Hierarchy

The main hierarchy of alphaTab starts of with the `alphaTab.model.Score` class. From there a hierarchical structure is going down to the individual note:

Score > Track > Staff > Bar > Voice > Beat > Note

Some general facts about the data model:

- Each hierarchy level provides usually bidirectional navigation within the hierarchy. e.g. from a note you can navigate fully up to the score.
- Many hierarchy level provide a dual-linked-list mechanism to navigate back and forth in time. e.g. from a beat you can navigate to the next beat of the same voice and track.
- The score provides a `finish` method which consolidates many aspects of the data model like resolving tied notes, calculating times and linking elements.

Score

The `Score` class contains some high-level information like the song title, artist, copyright holder and main tempo. It also contains a `stylesheet` property which is the only place in the data model where potentially some visual information is stored. Some file formats like Guitar Pro give some additional hints on how to render the music sheet.

Example

A score is one whole file loaded (e.g. Guitar Pro or MusicXML) where all the individual instruments are inside.

Score

Guitar Standard Tuning

The image displays a musical score for three tracks: Vocals, Guitar, and TAB. The tempo is marked as 120 BPM. The time signature is 3/4. The Vocals track features a melody with three measures, each starting with a red number (1, 2, 3) above the first note. The Guitar track plays a rhythmic accompaniment of eighth notes, marked with a forte 'f' dynamic. The TAB track shows the corresponding fret numbers (1 and 3) for the guitar part.

rendered by alphaTab

MasterBar (Score.masterBars)

The `MasterBar` items stored in `score.masterBars` contain information which is valid across all tracks for a particular bar or measure.

This includes information like time signatures, key signatures or repeats. alphaTab expects all bars of all tracks to follow the same playback sequence. Pieces where maybe the first instrument is repeating the first bar 5 times while the second instrument is continuing.

This is needed to consistently align the music notation elements when rendering multiple tracks at the same time, or when handling the audio playback.

Example

When you have a music piece which has e.g. 5 bars (aka. measures) and 3 instruments, the master bar 1 describes the details about the first bar of all 3 instruments.

$\text{♩} = 120$

MasterBar 1 **MasterBar 2** **MasterBar 3**

Vocals

Guitar

f

f

TAB

The image shows a musical score for three tracks: Vocals, Guitar, and TAB. The score is divided into three MasterBars: MasterBar 1 (red border), MasterBar 2 (blue border), and MasterBar 3 (green border). MasterBar 1 contains measures 1-4, MasterBar 2 contains measures 5-6, and MasterBar 3 contains measures 7-8. The tempo is 120 BPM. The Vocals track is in 3/4 time for MasterBar 1 and 3, and 2/4 time for MasterBar 2. The Guitar track is in 3/4 time for MasterBar 1 and 3, and 2/4 time for MasterBar 2. The TAB track shows fret numbers for each measure. The first track (Vocals) has a dynamic marking of *f* (forte). The second track (Guitar) also has a dynamic marking of *f* (forte).

MasterBar	Measure	Vocals	Guitar	TAB
MasterBar 1	1	C4	C4, E4, G4	3, 3, 1
	2	D4	C4, E4, G4	3, 3, 1
	3	E4	C4, E4, G4	3, 3, 1
	4	F4	C4, E4, G4	3, 3, 1
MasterBar 2	5	G4	C4, E4, G4	3, 3, 1
	6	A4	C4, E4, G4	3, 3, 1
MasterBar 3	7	B4	C4, E4, G4	3, 3, 1
	8	C5	C4, E4, G4	3, 3, 1

Track (Score.tracks)

A `Track` (aka. Part) represents a single instrument of the overall music piece. Like the lead guitar or the drums.

The track has a name and defines some playback information via the `playbackInfo` property. The `playbackInfo` of the track contains info like the midi channels and program used for playback.

The track contains on the next level the individual staves which contribute to the instrument.

Example

When you have a song with vocals and guitar, the vocals are one track and the guitar is one track.

♩ = 120

Track 1

Vocals

Track 2

Guitar

Staff (Score.tracks[].staves)

A `Staff` is the logical separation of the track into the individually played parts on the instrument. Like on a piano two separate staves might indicate what notes to play on the left and right hand.

This staff object in the data model must not be confused with a "visual" staff when the music sheet is rendered. A single staff in the data model, might result in 2 visual staves being rendered: one staff for the standard notation, a second staff for the guitar tabs.

The staff is again just a container for the individual bars of the song. It contains some information about which visual staves to show or the string tuning for fretted instruments.

Example

In the following example you can see both kinds of staff representations visually. For the first track we have two individual staves in the data model. Each staff of the first track has own individual notes inside. On the second track there is only one staff in the data model but still it is displayed as two visual staff representations. Both visual staves of the second

track have the same notes, but just with a different display (standard notation or guitar tabs).

Piano

Track 1 - Staff 1

Track 1 - Staff 2

Guitar

Track 2 - Staff 1

Track 2 - Staff 2

Bar (`Score.tracks[].staves[].bars`)

A `Bar` (aka. measure) is another container to bring us closer to the individual notes. Bars split the staff into segments separated with vertical lines when rendered.

The bar itself defines some basics like the clef shown or if simile marks indicate a repeats. The list of bars is ordered in the timewise / indexed fashion.

The bars contain voices as a next level.

Example

The example shows a musical score for Piano and Guitar. The tempo is marked as 120. The Piano part consists of three bars: Bar 1 (3/4 time, quarter notes), Bar 2 (2/4 time, quarter notes), and Bar 3 (3/4 time, quarter notes). The Guitar part consists of three bars: Bar 1 (3/4 time, chords), Bar 2 (2/4 time, chords), and Bar 3 (3/4 time, chords). The Guitar part also includes a TAB section with fret numbers (3, 3, 3, 3, 3, 3, 1, 1, 1, 1, 3, 3, 1, 1, 3, 3, 1, 1).

Voice (Score.tracks[].staves[].voices[])

A `Voice` allows to have multiple independent notes placed within the same bar indicating the simultaneous playing of them with different duration.

The voice is again just a container for the individual beats played in it. Multiple voices are usually only used for certain types of instruments where this kind of display and play type is possible and common. It is similar to the concept of multiple staves. Voices can also be seen as multiple staves squashed into one.

This list is ordered in the fashion that the first item is the primary voice, and others are considered additional ones.

Example

By default alphaTab renders the primary voices fully black and additional voices as semi transparent.

pno.

$\text{♩} = 120$

Bar 1 Voice 2

Bar 2 Voice 2

Bar 1 Voice 1

Bar 2 Voice 1

Beat (`Score.tracks[].staves[].voices[].beats`)

A `Beat` (aka. Chord) group multiple notes played at the same time with the same length.

This is the first level here we have a bit more information about the actual music notation like: the durations, tuplets, lyrics, whammy bar effects, chord diagrams, grace notes, etc. This list is ordered in a timewise fashion from left to right.

The notes will ultimately contain the individual notes.

Example

Voice 2 Beat 2

Voice 2 Beat 3

Voice 2 - Beat 1

Voice 2 Beat 4

s.guit.

Voice 1 Beat 1

Voice 1 Beat 2

Note (Score.tracks[].staves[].voices[].beats[].notes)

A **Note** represents a single musical sound played by the instrument.

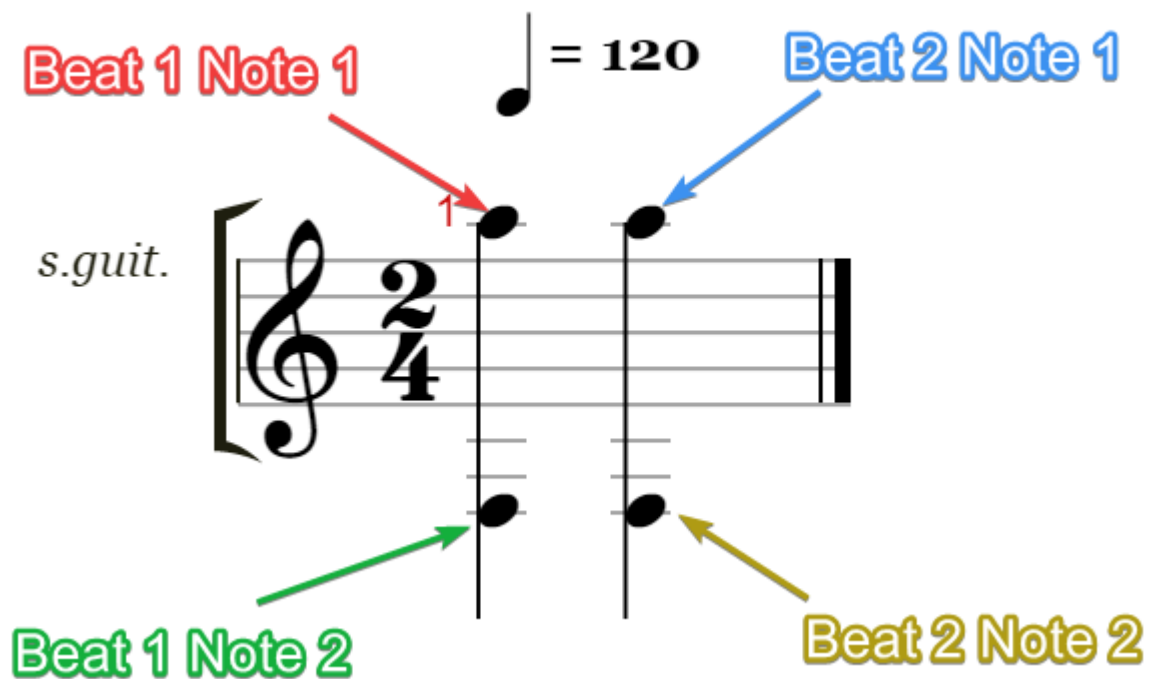
In alphaTab there can be 3 types of notes:

1. Fretted notes where the note pitch is determined by combining the string tuning of the staff, and the fret played on a particular string of the instrument.
2. Piano notes where the note pitch is determined by combining an octave and the tone within this octave.
3. Percussion notes where the a combination of the percussion element and a variation indicate what to play.

The note list is not ordered in any particular fashion.

Notes have a huge set of annotations and effects which can influence the display and playback like:

- Hammer-Ons/Pull-Offs
- Harmonics
- Accentuations
- Bends
- Palm Mutes / Let Rings
- Slides
- and much more...



Example

Stylesheet

alphaTab is moving gradually to a stylesheet based configuration of how music sheets should be displayed. Many file formats provide options on what to display and how and alphaTab tries to respect this information. The related settings are accessible via `Score.stylesheet`.

The `Settings` related to display will be merged over time into the stylesheet. We envision to define a default stylesheet and overrides on `Settings` level.

The stylesheet doesn't have yet a reference documentation pages here hence the documentation on code level is the best reference:

<https://github.com/CoderLine/alphaTab/blob/develop/src/model/RenderStylesheet.ts>

[✎ Edit this page](#)